

Incremental Multi-Agent Path Repair for Dynamic Automated Warehouses

Course: Autonomous Systems

Group member	Roll number
Jatin Purbia	B23CH1022
Sharana Gouda Lakshetty	B23CH1042

Abstract

We present a prototype automated-warehouse simulator in which a fleet of robots moves on a 2D grid, picks up and delivers parcels, and plans collision-free routes with Space-Time A* under a prioritized multi-agent scheme. When the environment changes at run time – a cell is suddenly blocked, a robot breaks down, or a robot is assigned a high-priority emergency task – the system repairs the affected plans *incrementally*: only the directly disrupted robots are touched first, and a Contract-Net-style negotiation protocol lets them ask immediate neighbours to yield before any larger set of plans is disturbed. A full from-scratch replan is kept only as an experimental baseline, never as part of the normal repair path.

We evaluate the approach over 5 agent counts $\times$ 5 obstacle densities $\times$ 20 seeds $\times$ 3 strategies (single-agent repair, the proposed local negotiated repair, and global replanning), totalling 6000 simulation runs across four disruption scenarios. The local negotiated strategy modifies no more agents than global replanning in 100% of matched runs (11.97 vs. 13.98 modified agents on average for cell blockage) while using roughly a quarter of the repair time, at the cost of a flowtime that is on average about 10% higher. Negotiation makes the difference between the proposed method and a naive single-agent repair: the latter fails on 52 of 500 cell-blockage runs (up to 70% failures at 40 agents and density 0.20), while the negotiated version resolves nearly all of them at a few milliseconds of extra cost.

Contents

1	Introduction	3
2	Problem formulation	4
3	Related concepts	4
4	Warehouse environment	5
5	Initial Space-Time A* planning	6
5.1	Search state and heuristic	6
5.2	Successors and legality	6
5.3	Reservation table	7
5.4	Initial multi-agent planning	7
6	Dynamic disruption model	8
7	Proposed incremental repair algorithm	8
7.1	The six strategies, in order of preference	9
7.2	Worked example	9
8	Neighbour negotiation protocol	9
9	Experimental methodology	10
10	Metrics	11
11	Results	12
11.1	Success rate vs. agent count and density	12
11.2	Modified agents and flowtime (strategy B)	12
11.3	Repair time (strategy B vs. C)	13
11.4	Paired comparison: B (local) vs. C (global)	13
11.5	Results by hypothesis	13
11.6	Local negotiated repair vs. single-agent repair	14
12	Discussion	14
13	Limitations	15
14	Conclusion	15
A	Repository layout	16
B	Test summary	16
C	Repair configuration parameters	17
D	Full result tables	17

1 Introduction

Automated warehouses increasingly rely on fleets of mobile robots that must navigate a shared grid floor without colliding, while continuously picking up and delivering parcels. Multi-agent pathfinding (MAPF) algorithms such as prioritized planning, Conflict-Based Search (CBS) and Space-Time A* can compute a collision-free initial plan for such a fleet, but real warehouses are not static: a shelf blocks an aisle, a robot’s motor fails, or a rush order must be inserted ahead of everything else. Re-invoking the full multi-agent planner every time one of these events occurs is wasteful – it throws away work that is still valid for the vast majority of robots, and it is slow precisely when a fast reaction is most needed.

This report studies the alternative: *repair the plan instead of replacing it*. When a disruption occurs, only the robots whose own plan is actually invalidated are reconsidered first. If a repair for one of those robots would require displacing a robot whose plan is still valid, the two robots *negotiate* – modelled on the Contract-Net protocol – so that whichever side can absorb the change more cheaply does so, and the set of robots touched grows only as far as necessary. A full re-plan of every active robot is kept in the codebase purely as a baseline for comparison, never as the mechanism the system actually uses to respond to a disruption.

Research question

Can a dynamic multi-robot warehouse system repair its plans locally after a disruption, keeping the number of modified robots to a minimum, without resorting to full re-planning, and without a significant penalty in overall completion time?

Contributions

- A Space-Time A* planner with a shared reservation table and a chain heuristic over ordered pickup/delivery waypoints, used both for initial prioritized multi-agent planning and as the inner search routine of the repair algorithm.
- LOCAL_PLAN_REPAIR: a six-strategy incremental repair procedure (wait, temporal shift, local detour, single-agent replan, Contract-Net negotiation, group replan) that commits atomically and never touches a robot’s plan unless necessary.
- A bid-based decentralized negotiation protocol between a disrupted robot and the neighbours whose reservations block its ideal route, with an explicit cost function and message log.
- An experimental harness that sweeps agent count (5–40), dynamic-obstacle density (0–0.20), and three repair strategies over 20 seeds each, with reproducible CSV/JSON outputs and a web dashboard for visualizing the grid, the conflict graph, and the resulting metrics.

Roadmap

Section 2 formalizes the grid, robots, tasks and disruptions. Section 3 summarizes the MAPF and plan-repair concepts the prototype builds on. Section 4 describes the warehouse layout. Sections 5 to 8 describe the initial planner, the disruption model, the repair algorithm and the negotiation protocol. Section 9 describes the experimental methodology, Section 10 the metrics, Section 11 the results organized by hypothesis, Section 12 the discussion, Section 13 the limitations, and Section 14 the conclusion.

2 Problem formulation

Grid and time. The warehouse floor is a grid $G = (V, E)$ of cells; each cell is free or a static obstacle (a shelf). Time is discrete and synchronous: at every timestep t every active robot performs exactly one action, either a unit move to a 4-connected neighbour or a wait, each costing one timestep.

Robots and tasks. Each robot i starts at a cell $\text{pos}_i(0)$ and is assigned an ordered list of tasks, each a (pickup, delivery) cell pair; a robot carries at most one parcel, and pickup/delivery are instantaneous on arrival. A robot’s remaining work is therefore an ordered waypoint list; a robot leaves the workspace once its last delivery is complete, so finished robots never obstruct anyone else.

Plans. A plan π_i for robot i is a time-indexed sequence of positions. At any time t_d a plan decomposes as

$$\pi_i = \pi_i^{\text{executed}} \oplus \pi_i^{\text{remaining}},$$

where π_i^{executed} (times $\leq t_d$) is immutable and $\pi_i^{\text{remaining}}$ is what a repair may change.

Collision constraints. Two robots must never occupy the same cell at the same time (*vertex* conflict) and must never swap cells in one step (*edge* conflict). A robot must never occupy a cell that is a static obstacle, an active dynamic obstacle, or the cell a broken robot stands on.

Disruptions. Three disruption types are modelled:

- **Cell blockage:** a cell becomes impassable for an interval $[t_s, t_e]$ (or permanently, $t_e = \infty$), known in full as soon as it is announced.
- **Robot breakdown:** a robot stops moving at its current cell, which becomes a permanent obstacle; its unfinished tasks must be reassigned to other robots.
- **Emergency task:** a new task with elevated priority is inserted into some robot’s work.

Objectives. Two global performance metrics are reported: *flowtime* $\sum_i C_i$ (sum of completion times) and *makespan* $\max_i C_i$ (the total number of timesteps for every robot to finish, i.e. the quantity the assignment specification calls “total time steps required by all the agents to accomplish their respective tasks”). The repair objective itself is lexicographic:

$$J = W_1 \cdot |A_d| + W_2 \sum_i \Delta C_i + W_3 \cdot N_{\text{msg}} + W_4 \cdot T_{\text{repair}}, \quad W_1 \gg W_2 \gg W_3, W_4, \quad (1)$$

with $W_1 = 10000$, $W_2 = 100$, $W_3 = 1$, $W_4 = 0.1$ in the implementation, so that minimizing the number of modified agents $|A_d|$ dominates every other term.

3 Related concepts

The prototype implements, rather than cites a single paper for, the following standard multi-agent pathfinding (MAPF) concepts:

Space-Time A*

Ordinary A* search over a state space augmented with a time dimension, so that a single-agent path can be made to avoid a set of already-reserved space-time cells.

Prioritized planning

A MAPF algorithm in which robots are planned one at a time in priority order, each reserving its path

before the next robot is planned; fast but incomplete (a solvable instance can be reported as infeasible, e.g. a narrow corridor with two robots approaching head-on and no passing bay).

Reservation table

The shared data structure recording which (cell, time) and (edge, time) pairs are occupied, used both by the initial planner and by every repair search.

Plan repair vs. replanning

In *lifelong / online* MAPF settings, re-running a full MAPF solver after every environment change is correct but costly; *plan repair* instead patches only the plans that a change actually invalidates.

Contract-Net protocol

A classical multi-agent negotiation protocol (task announcement, competitive bidding, award) used here to decide, between a disrupted robot and the neighbours blocking its ideal route, who should actually move.

4 Warehouse environment

The default environment is a procedurally generated 20×14 shelf-and-aisle layout (`scenario_generator.warehouse_map`): rows of shelves (static obstacles) separated by aisles, with robots and task endpoints placed at random on free cells. Robots have capacity one (they can carry a single parcel at a time) and are assumed to leave the grid immediately after their final delivery, so completed robots never need to be modelled as obstacles for the rest of the simulation. When a cell blockage is announced, its entire interval $[t_s, t_e]$ is known at announcement time, which lets the planner choose to wait out a finite blockage rather than detour around it. Figure 1 shows the interactive simulation view: shelves, robots, pickup/delivery markers, dynamic obstacles, and planned/executed path overlays.



Figure 1. Simulation dashboard: grid layout, robots, a cell blockage, and the resulting repaired paths.

5 Initial Space-Time A* planning

5.1 Search state and heuristic

Each search state is a tuple $n = (x, y, t, q)$: a cell, a timestep, and q , the index of the next waypoint in the robot’s ordered task list. The cost so far is $g(n) = t - t_{\text{start}}$ (every move or wait costs one timestep). The heuristic is a *Manhattan chain* over the remaining waypoints,

$$h(n) = |x - x_q| + |y - y_q| + \sum_{k \geq q} |w_k - w_{k+1}|, \quad (2)$$

i.e. Manhattan distance to the next waypoint plus the Manhattan length of every remaining leg of the ordered waypoint chain. This heuristic never overestimates true remaining cost under 4-connected unit-cost moves and is consistent, so Space-Time A* returns a time-optimal path (subject to the current reservations) the first time a goal state is popped.

5.2 Successors and legality

From $n = (x, y, t, q)$ the four unit moves and a wait are generated. A step $u@t \rightarrow v@t+1$ is legal if and only if:

- (i) v is a traversable cell;
- (ii) v is not covered by a dynamic obstacle at $t+1$;
- (iii) $v@t+1$ is not already vertex-reserved by another robot; and
- (iv) no other robot is executing $v \rightarrow u$ in the same interval (the edge-swap check).

The waypoint index q only advances when the robot actually stands on waypoint w_q , which enforces pickup-before-delivery ordering; visiting a later waypoint’s cell early does not count. The search terminates when q reaches the end of the waypoint list. Failure modes are `unreachable` (a cheap BFS pre-check ignoring finite/temporary obstacles), `no_path` (the time horizon was exhausted) and `expansion_limit` (more than 250,000 states expanded).

A *soft-constraint* variant of the same search is used only during negotiation: stepping through a reservation held by a robot known to be willing to yield costs an extra penalty rather than being illegal, so the search returns the route that would disturb the fewest neighbours, and the neighbours actually crossed are read off as the candidate “blockers.”

Algorithm 1 Space-Time A* (single robot, hard constraints)

```

1: function SPACETIMEASTAR(robot, waypoints, reservations,  $t_{\text{start}}$ )
2:   open  $\leftarrow \{(x_0, y_0, t_{\text{start}}, 0)\}$ ;  $g \leftarrow 0$ ;  $f \leftarrow h(\text{start})$ 
3:   while open not empty do
4:      $n \leftarrow$  pop state with minimal  $f$ 
5:     if  $n.q = |\text{waypoints}|$  then
6:       return reconstructed path
7:     end if
8:     for each of the 4 moves and the wait from  $n$  do
9:        $n' \leftarrow$  successor at  $t+1$ 
10:      if  $n'$  violates an obstacle, a vertex reservation, or an edge-swap then
11:        skip
12:      end if
13:      if  $n'$  stands on waypoint  $w_{n.q}$  then
14:         $n'.q \leftarrow n.q + 1$ 
15:      end if
16:      relax  $g(n')$ ; push with  $f(n') = g(n') + h(n')$ 
17:    end for
18:  end while
19:  return failure (unreachable / no_path / expansion_limit)
20: end function

```

5.3 Reservation table

The reservation table stores vertex reservations $R_V(v, t)$ (robot occupies v at t) and edge reservations $R_E(u, v, t)$ (robot moves $u \rightarrow v$ during $[t, t+1]$); a move $u \rightarrow v$ is rejected if $R_E(v, u, t)$ already exists. A robot *following* another into a cell it vacates in the same step is allowed; swapping cells or sharing a cell is not. `release_robot(rid, after_time)` removes only the reservations after a given time, so a robot's already-executed prefix is never disturbed when its future is released for repair.

5.4 Initial multi-agent planning

The full fleet is planned once, at $t = 0$, by *prioritized planning*: robots are planned one at a time in descending `planning_priority`, and each robot's complete path is reserved in the shared table before the next robot is planned. If some robot cannot be planned against the current reservations, up to 5 deterministic alternative priority orders are tried (the failed robot is bumped to the front, then a seeded shuffle is tried). This initial planner is used exactly once, to produce the starting plan; it is never invoked again as part of ordinary disruption handling.

Prioritized planning is known to be incomplete – e.g. two robots approaching head-on in a one-cell-wide corridor with no passing bay can be a genuinely unsolvable priority order even though a solution exists for a different order or a full MAPF solver – and the engine reports such instances as a planning failure rather than ever producing colliding plans.

6 Dynamic disruption model

For each disruption the *directly invalidated set* A_d^0 is the set of robots whose already-computed, not-yet-executed plan is incompatible with the new fact.

Cell blockage. A $\text{DynamicObstacle}(v, t_d, t_d + \text{duration})$ is added to the map. A_d^0 is every robot whose unexecuted plan steps onto v while it is blocked.

Robot breakdown. The broken robot stops on its current cell, which becomes a permanent obstacle owned by it; its executed prefix is kept, its future plan is dropped, and it is counted as “modified” (its own plan was truncated). Every one of its unfinished tasks returns to a task pool and is reassigned to

$$a^* = \arg \min_i d(\text{pos}_j, \text{pickup}) + d(\text{pickup}, \text{delivery}) + \lambda \cdot \text{Load}(a_j) + \mu \cdot \text{RepairImpact}(a_j), \quad (3)$$

with $\lambda = 4$, $\mu = 2$ by default, where $\text{RepairImpact}(a_j)$ counts how many *other* robots’ remaining plans pass through the bounding box spanned by $\{\text{pos}_j, \text{pickup}, \text{delivery}\}$ – i.e. reassignment prefers a recipient that will disturb few other robots, not merely the geometrically nearest one. If the broken robot was already carrying a parcel, the handover point is the free cell nearest to the delivery. Recipients keep their existing plan and, where possible, only the *extension* covering the new waypoints is planned; a full replan of the recipient is used only if the extension is infeasible. A_d^0 is the set of recipients plus every robot whose plan passes through the now-blocked breakdown cell.

Emergency task. The robot minimizing the same cost expression (3) (or a UI-selected robot) receives the new task, inserted immediately after its currently-carried parcel (respecting capacity one) or before its first pending task. Its `planning_priority` is raised by 1000 and it is flagged `emergency`; because the insertion changes its waypoint order, its plan is always fully recomputed rather than patched.

7 Proposed incremental repair algorithm

LOCAL_PLAN_REPAIR (Algorithm 2) is the single procedure invoked for every disruption under the proposed strategy. Crucially, every robot outside the (initially small, possibly growing) affected set A keeps its reservations and its plan untouched throughout.

Algorithm 2 LOCAL_PLAN_REPAIR(d, t_d)

- ```

1: update the dynamic map with disruption d
2: $A \leftarrow$ directly affected agents A_d^0 ; keep every robot's executed prefix
3: release future reservations of A only $\triangleright$ all other reservations stay fixed
4: for each pending agent $a \in A$, emergency robots first, then by priority do
5: candidate $\leftarrow$ first validating option of: (1) wait, (2) temporal shift,
 (3) local detour / single-agent replan
6: if candidate exists and a is not an emergency robot and added delay ≤ 12 then
7: commit candidate
8: else
9: NEGOTIATE(a) $\triangleright$ Section 8
10: end if
11: end for
12: validate every committed plan against every other plan and every obstacle
13: commit atomically, or fail and leave every plan as it was

```

## 7.1 The six strategies, in order of preference

1. **Wait** – insert between 1 and 12 waits immediately before the first illegal step of the plan.
2. **Temporal shift** – delay the entire remaining plan by 1–12 steps.
3. **Local detour** – replace a bounded window of the plan (at most 14 steps) with a fresh Space-Time A\* leg that rejoins the original plan; waypoints inside the window are still visited. If no rejoining window works, a full single-agent replan over the remaining waypoints is attempted instead.
4. **Priority exchange** – during negotiation or group repair, the cheaper-to-move robot yields even when it nominally has a higher priority (logged as a PRIORITY\_UPDATE message); during group replanning a robot that cannot be placed is bumped to the front of the planning order.
5. **Expansion** –  $A \leftarrow A \cup \{a_j\}$  for a specific neighbour  $a_j$ ; only  $a_j$ 's future reservations are released, nothing else.
6. **Group replan** – prioritized Space-Time A\* run over exactly the (small, expanded) affected group, with every other robot's plan held fixed.

Every candidate at every strategy level is validated against the full set of obstacles and all other robots' fixed reservations before it is accepted, and the entire repair is committed atomically: if validation ever fails, no partially-applied plan is ever left in place and the run is recorded as unsuccessful rather than silently producing a collision.

## 7.2 Worked example

A single cell blockage ahead of one robot that can simply wait it out produces  $|A_d| = 1$  (strategy 1, no negotiation). A blockage that forces a detour through a cell another robot's plan also needs produces  $|A_d| = 2$ : the disrupted robot's detour triggers one negotiation round, the single blocking neighbour yields with its own local detour, and the repair commits with two modified plans and one REPAIR\_COMMITTED message. This scenario is reproducible with

```
python -m app.cli run s2-negotiated-repair --events
```

which prints the full event log.

## 8 Neighbour negotiation protocol

When a disrupted robot  $a$  cannot be repaired by strategies 1–3 alone (or is itself an emergency robot, or its own fallback is expensive), it negotiates with its neighbours rather than being folded into an uncontrolled replan:

1. **Ideal route.**  $a$  computes its own *ideal* route using the soft-constraint search of Section 5. The robots whose reservations this ideal route crosses are the *blockers*, ordered by time of first conflict.
2. **Request.** For each blocker  $j$ ,  $a$  sends a REPAIR\_REQUEST; each blocker attempts to yield (strategies 1–3) against a table that already contains  $a$ 's ideal route.
3. **Bidding.** Both sides submit bids

$$\text{Bid}_i = w_1 \Delta C_i + w_2 \Delta L_i + w_3 P_i + w_4 M_i, \quad (4)$$

with  $w_1 = 1$ ,  $w_2 = 0.5$ ,  $w_3 = 1$ ,  $w_4 = 30$ . Here  $\Delta C_i$  is the increase in completion time,  $\Delta L_i$  the extra path length,  $P_i$  a priority penalty ( $P_i = \text{priority}_i + 1000$  for an emergency robot, making it extremely

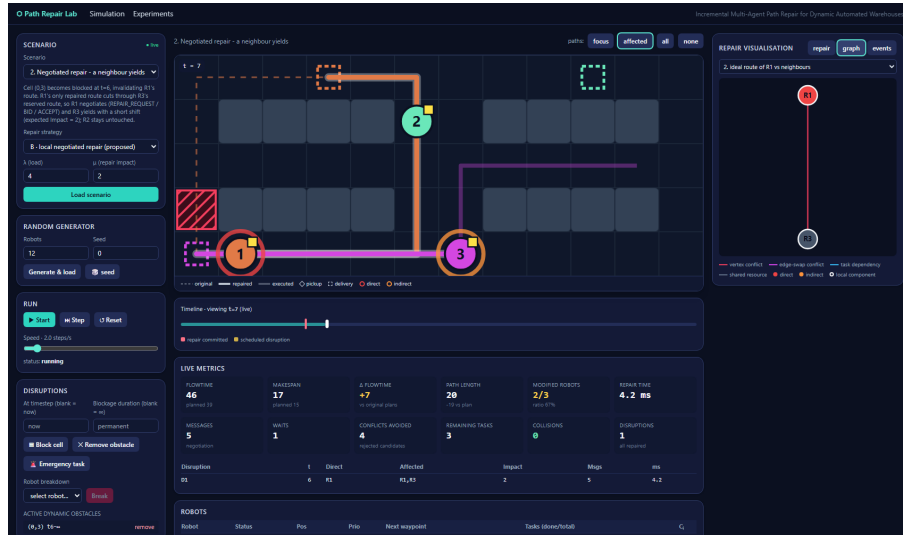
- costly to delay one), and  $M_i = 1$  exactly when the robot has not yet had its plan modified (so bringing in a new, previously-unmodified robot costs an extra  $w_4$ ). A robot that cannot yield locally bids  $\infty$ .
4. **Award.** If  $a$ 's own fallback bid is no worse than the sum of the blockers' bids,  $a$  yields and the blockers are sent REJECT; otherwise the blockers are sent ACCEPT, their new plans are adopted, they join  $A$  (an `AFFECTED_SET_EXPANDED` event is logged), and a `PRIORITY_UPDATE` records any priority inversion or emergency yielding.
  5. **Group escalation.** If a blocker cannot yield locally,  $a$  and its blockers form a *group* and strategy 6 (group replan) is attempted on that group alone. If the group replan itself fails, the group expands to include the neighbour with the cheapest expansion cost

$$w_4 + P_j - 5 \cdot (\text{\#conflicts with the group's ideal routes})$$

- (or the nearest robot, if no conflicts are visible), repeating until either a group replan succeeds or every active robot is included, at which point the repair is reported as a failed / deadlocked run.
6. **Commit.** A final `REPAIR_COMMITTED` message is sent to every modified robot except the initiator. Robots that only exchanged negotiation messages but ended up with an unchanged plan are *not* counted as modified – the impact set is always computed from actual plan differences, never from message traffic.

If a robot is already committed earlier in the same repair and a later robot in the same event finds no feasible route, already-repaired robots can be re-opened and folded into the growing group; this was needed for completeness, since without it some instances that global replanning could solve were being reported as local-repair failures.

All of this message traffic – `REPAIR_REQUEST`, `BID`, `ACCEPT/REJECT`, `PRIORITY_UPDATE`, `AFFECTED_SET_EXPANDED`, `REPAIR_COMMITTED` – is recorded on a message bus and is both counted as the  $N_{\text{msg}}$  metric and rendered in the dashboard's repair inspector and conflict-graph views (Figure 2).



**Figure 2.** Conflict graph view: nodes are robots, edges are vertex/edge-swap/shared-resource/dependency conflicts; the connected component of the disrupted robot is highlighted.

## 9 Experimental methodology

Table 1 summarizes the experimental configuration.

**Table 1.** Experimental configuration.

| Item                                      | Value                                                                                        |
|-------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Layout</b>                             | $20 \times 14$ shelf-and-aisle warehouse, 1 task per robot, random free endpoints            |
| <b>Agent counts <math>N</math></b>        | 5, 10, 20, 30, 40                                                                            |
| <b>Obstacle density <math>\rho</math></b> | 0, 0.05, 0.10, 0.15, 0.20 (fraction of traversable cells blocked)                            |
| <b>Blockage placement</b>                 | 50% targeted onto planned paths (guaranteed to matter), 50% uniformly random                 |
| <b>Blockage duration</b>                  | 6–14 timesteps                                                                               |
| <b>Disruption types</b>                   | cell_blockage; robot_breakdown; emergency_task; mixed (all three)                            |
| <b>Repetitions</b>                        | 20 seeds (0–19) per $(N, \rho)$ pair                                                         |
| <b>Strategies</b>                         | A: single-agent repair; B: local negotiated repair (proposed); C: global replan (baseline)   |
| <b>Total runs</b>                         | $5 \times 5 \times 20 \times 3 = 1500$ per disruption type; 6000 overall                     |
| <b>Pairing</b>                            | all three strategies see the identical scenario and disruption list for a given seed         |
| <b>Statistics</b>                         | mean $\pm$ sample standard deviation over <i>successful</i> runs; success rate over all runs |
| <b>Hardware</b>                           | 12 logical CPUs, Python 3.12.4, an 8-process pool                                            |
| <b>Collisions observed</b>                | 0 in all 6000 runs (the engine asserts vertex/edge collisions as hard errors)                |

All strategies for a given seed face the exact same randomly generated scenario and the exact same sequence of disruptions, so differences between strategies are attributable to the repair mechanism rather than to scenario variance. Experiments are reproduced with:

```
cd backend
for t in cell_blockage mixed robot_breakdown emergency_task; do
 python -m app.experiments.batch_runner --type $t --reps 20 --seed 0 \
 --out experiment_outputs --name default_$t
 python -m app.experiments.make_tables experiment_outputs/default_$t.json \
 > ../docs/results/$t.md
done
```

## 10 Metrics

### Flowtime and makespan

$\sum_i C_i$  and  $\max_i C_i$  – the total-timesteps metric requested by the assignment.

### Impact / impact ratio

The count, respectively the fraction, of robots whose committed plan actually differs from before the disruption,  $\text{modified\_agents}(\pi, \pi') = \{i : \pi_i \neq \pi'_i\}$ ; message-only participants are excluded by construction.

### $\Delta$ flowtime

Change in flowtime relative to the originally planned (pre-disruption) flowtime.

### Repair runtime and message count

Runtime (ms) and number of messages – the cost of the repair mechanism itself.

### Secondary repair statistics

Additional path length, largest  $|A_d|$  of any single repair, and affected-set expansions per run.

### Success rate

Share of runs in which every task is completed.

## Collisions and deadlocks

Both asserted as hard invariant violations by the engine; zero were observed.

A robot’s completion time, and all metrics, are computed from *executed* trajectories, not from plans, and a broken robot’s completion time is defined as its breakdown time.

## 11 Results

Unless noted, all numbers are for the `cell_blockage` disruption type; the other three disruption types (`robot_breakdown`, `emergency_task`, `mixed`) show the same qualitative pattern (see `docs/results/*.md`). In every grid table, rows are agent counts  $N$  and columns are obstacle densities  $\rho$ .

### 11.1 Success rate vs. agent count and density

**Table 2.** Success rate (% of 20 seeds completing every task), shown as strategies A/B/C.

| $N$ | Obstacle density $\rho$ |             |            |             |            |
|-----|-------------------------|-------------|------------|-------------|------------|
|     | 0                       | 0.05        | 0.10       | 0.15        | 0.20       |
| 20  | 100/100/100             | 95/100/100  | 95/100/100 | 100/100/100 | 85/95/100  |
| 30  | 100/100/100             | 100/100/100 | 85/100/100 | 75/100/100  | 70/100/100 |
| 40  | 100/100/100             | 80/100/100  | 70/100/100 | 65/100/100  | 30/100/100 |

### 11.2 Modified agents and flowtime (strategy B)

**Table 3.** Mean modified agents per run, strategy B, over successful runs (mean  $\pm$  s.d.).

| $N$ | Obstacle density $\rho$ |                |                |                |                |
|-----|-------------------------|----------------|----------------|----------------|----------------|
|     | 0                       | 0.05           | 0.10           | 0.15           | 0.20           |
| 5   | 0.0                     | 4.0 $\pm$ 1.0  | 4.5 $\pm$ 0.7  | 4.9 $\pm$ 0.2  | 5.0 $\pm$ 0.0  |
| 10  | 0.0                     | 6.3 $\pm$ 1.3  | 8.2 $\pm$ 1.3  | 9.3 $\pm$ 0.9  | 9.7 $\pm$ 0.5  |
| 20  | 0.0                     | 10.6 $\pm$ 2.2 | 14.6 $\pm$ 2.1 | 17.3 $\pm$ 1.5 | 18.4 $\pm$ 1.3 |
| 30  | 0.0                     | 13.1 $\pm$ 2.2 | 20.2 $\pm$ 2.2 | 24.1 $\pm$ 1.9 | 26.0 $\pm$ 1.9 |
| 40  | 0.0                     | 15.2 $\pm$ 2.9 | 25.6 $\pm$ 3.2 | 29.6 $\pm$ 3.6 | 32.5 $\pm$ 3.3 |

**Table 4.** Mean flowtime (timesteps), strategy B, over successful runs (mean  $\pm$  s.d.).

| $N$ | Obstacle density $\rho$ |               |               |               |               |
|-----|-------------------------|---------------|---------------|---------------|---------------|
|     | 0                       | 0.05          | 0.10          | 0.15          | 0.20          |
| 5   | 111 $\pm$ 22            | 150 $\pm$ 24  | 163 $\pm$ 26  | 177 $\pm$ 23  | 189 $\pm$ 27  |
| 10  | 230 $\pm$ 23            | 283 $\pm$ 30  | 313 $\pm$ 34  | 336 $\pm$ 31  | 361 $\pm$ 34  |
| 20  | 478 $\pm$ 39            | 549 $\pm$ 42  | 598 $\pm$ 40  | 640 $\pm$ 39  | 694 $\pm$ 63  |
| 30  | 725 $\pm$ 46            | 807 $\pm$ 56  | 885 $\pm$ 62  | 946 $\pm$ 61  | 990 $\pm$ 61  |
| 40  | 961 $\pm$ 56            | 1044 $\pm$ 59 | 1152 $\pm$ 71 | 1217 $\pm$ 71 | 1294 $\pm$ 85 |

### 11.3 Repair time (strategy B vs. C)

**Table 5.** Mean repair time (ms) over successful runs, shown as B (local)/C (global).

| $N$ | Obstacle density $\rho$ |         |          |          |          |
|-----|-------------------------|---------|----------|----------|----------|
|     | 0                       | 0.05    | 0.10     | 0.15     | 0.20     |
| 5   | 0/0                     | 10/107  | 19/177   | 28/277   | 43/358   |
| 10  | 0/0                     | 21/236  | 59/405   | 77/603   | 108/773  |
| 20  | 0/0                     | 66/445  | 175/773  | 218/1168 | 282/1398 |
| 30  | 0/0                     | 158/636 | 290/1166 | 431/1887 | 569/2299 |
| 40  | 0/0                     | 254/904 | 514/1633 | 731/2274 | 917/3071 |

### 11.4 Paired comparison: B (local) vs. C (global)

**Table 6.** Paired comparison over runs where both B and C succeeded, by disruption type.

| Disruption type | Pairs | Modified agents<br>B / C | Repair time (ms)<br>B / C | Pairs where C has<br>lower flowtime |
|-----------------|-------|--------------------------|---------------------------|-------------------------------------|
| cell_blockage   | 498   | 11.97 / 13.98            | 199 / 823                 | 79.9%                               |
| mixed           | 421   | 12.5 / 14.8              | 226 / 936                 | 89.5%                               |
| robot_breakdown | 439   | 12.3 / 14.3              | 287 / 1103                | 88.4%                               |
| emergency_task  | 498   | 12.5 / 15.1              | 565 / 2426                | 86.1%                               |

B modified no more agents than C in 100% of the 498 cell-blockage pairs (and  $\geq 99.8\%$  across the other types).

### 11.5 Results by hypothesis

#### H1 – More robots lead to more congestion, flowtime, repair cost and modified agents.

*Supported*, with a trivial component: flowtime is a sum over robots, so it grows with  $N$  by construction. Non-trivially, at  $\rho = 0.10$  repair time for strategy B grows  $19 \rightarrow 59 \rightarrow 175 \rightarrow 290 \rightarrow 514$  ms for  $N = 5, \dots, 40$ , and modified agents grow  $4.5 \rightarrow 8.2 \rightarrow 14.6 \rightarrow 20.2 \rightarrow 25.6$ . The modified *ratio* actually falls with  $N$  ( $0.80 \rightarrow 0.38$  at  $\rho = 0.05$ ), because the absolute number of blockages scales with  $\rho$ , not with  $N$ .

#### H2 – More obstacles lead to more detours, repair time, and failures.

*Partly supported*. Flowtime and repair time rise monotonically with  $\rho$  for all three strategies. Failure rate rises sharply for strategy A ( $N = 40$ :  $100\% \rightarrow 80\% \rightarrow 70\% \rightarrow 65\% \rightarrow 30\%$  success as  $\rho$  increases) but *not* for B (only 2 failures in 500 cell-blockage runs) or C (zero failures).

#### H3 – Local negotiated repair modifies fewer agents than global replanning.

*Supported*: 11.97 vs. 13.98 modified agents on average, with  $B \leq C$  in 100% of 498 pairs; the gap is largest at  $N = 40$ ,  $\rho = 0.05$  (15.2 vs. 25.9).

#### H4 – Global replanning may occasionally give a lower flowtime, at the cost of more repair time and

### more plan disruption.

*Supported, but “occasionally” understates it:* C has lower flowtime in roughly 80–90% of matched pairs (about 10% lower on average) at roughly  $4\times$  the repair time. Part of this advantage is a confound – global replanning also re-optimizes the waits already present in the original prioritized plan, not only the effect of the disruption itself.

### H5 – Local repair is most beneficial relative to global replanning at low/moderate density.

*Supported for the modified-agent gap:* at  $N = 40$  the gap  $C - B$  shrinks as density rises – 10.7 ( $\rho = 0.05$ ), 7.9 ( $\rho = 0.10$ ), 5.9 ( $\rho = 0.15$ ), 4.0 ( $\rho = 0.20$ ) – because at high density with few robots almost every robot’s plan is touched by either strategy regardless.

### H6 – At high congestion the affected component expands to include many robots.

*Not supported* in this setup: affected-set expansions are rare ( $\leq 1.3$  per run even at  $N = 40$ ,  $\rho = 0.20$ ) and the largest single-repair  $|A_d|$  observed is about 6 of 40 robots. Most of the accumulated “modified agents per run” total comes from many separate, small, independent blockages each hitting a different robot – not from a single repair cascading through the fleet. Message counts for strategy B do grow with both  $N$  and  $\rho$ , but driven by the number of separate disruptions, not by expansion size.

## 11.6 Local negotiated repair vs. single-agent repair

B and A modify essentially the same number of agents when A succeeds (10.37 vs. 10.37 over 448 matched pairs) and achieve the same flowtime. The value negotiation adds is *feasibility*: A fails outright whenever a conflict with an unmodified robot’s plan arises (52 failures out of 500 cell-blockage runs, rising to 70% failure at  $N = 40$ ,  $\rho = 0.20$ ), whereas B resolves nearly all of these cases through negotiation, at a cost of only a few extra milliseconds of repair time.

## 12 Discussion

The headline trade-off is: local negotiated repair (B) touches fewer robots and runs roughly  $4\times$  faster than global replanning (C), at the price of a flowtime that is on average about 10% higher in the 80–90% of cases where C actually wins on flowtime. For a real warehouse, where minimizing disruption to already-in-progress robots (and the operational cost of recomputing and redistributing many new plans) matters as much as raw throughput, this trade is favourable.

The affected set only grows beyond the directly invalidated robots in a small minority of repairs (affected-set expansions  $\leq 1.3$  per run even in the most congested setting tested), which means the negotiation mechanism is doing its job: most of the time a disrupted robot’s neighbours can absorb the change locally without the disturbance propagating further. The negotiation weight  $w_4 = 30$  (the penalty for touching a previously-unmodified robot) is the main knob controlling this behaviour; informally, raising it would make the system even more conservative about expanding the affected set at the cost of occasionally failing to find any local repair, while lowering it would let cheaper but more widely-distributed repairs take over from the self-contained ones observed here.

Failures of both B and C are dominated by genuinely infeasible local instances (“no feasible route even if neighbours yield” / “group unsolvable”), and for breakdowns specifically by a broken robot stopping on another robot’s pickup or delivery cell or in a position that walls off a route. These are inherent to the generated instances and are not evidence favouring one strategy over another (B: 2 such failures in 500 cell-blockage runs; C: 0; breakdown-specific failures: 42 for B vs. 45 for C).

## 13 Limitations

- Prioritized planning (both for the initial plan and for group repair) is incomplete: a solvable instance can be reported as a failure, and a repair failure simply ends the run rather than triggering a partial recovery.
- Robots are assumed to leave the grid at their final delivery; parked robots that could later be re-blocked in place are not modelled, which keeps repair instances somewhat easier than a warehouse where idle robots remain on the floor.
- A broken robot stopping on another robot’s pickup/delivery cell makes that task permanently unreachable and the repair fails; blockage intervals must be known in full at the moment they are announced.
- A broken robot is itself counted as a modified agent (its plan is truncated), and its completion time is defined as its breakdown time.
- $\Delta$  flowtime is always measured against the *originally* planned flowtime; for breakdown runs this conflates the effect of lost/reassigned tasks with the effect of the repair mechanism itself.
- “Conflicts avoided” counts tentative repair candidates rejected during validation, not physical near-misses between robots.

**Threats to validity.** A single synthetic layout and grid size; one task per robot; a single blockage-duration range; a fixed targeted-blockage fraction of 0.5. Means are computed over successful runs only, so a strategy that selectively fails on hard instances can look artificially better on flowtime – the paired tables and the success-rate table must be read together, not in isolation. Timings are wall-clock on one machine in a process pool and should be read as relative, not absolute.

## 14 Conclusion

We built and evaluated a dynamic multi-robot warehouse simulator in which Space-Time A\* under a shared reservation table produces the initial fleet plan, and a six-strategy incremental repair procedure – wait, shift, detour, single-agent replan, Contract-Net negotiation, and bounded group replan – handles cell blockages, robot breakdowns and emergency tasks without ever invoking a full re-plan as part of its normal operation. Across 6000 simulation runs spanning 5–40 agents and obstacle densities up to 0.20, the proposed local negotiated strategy modified no more robots than a full global replan in every matched run, used roughly a quarter of its repair time, and recovered almost every case where a naive single-agent-only repair (no negotiation) would simply have failed. The main cost is a flowtime that is on average about 10% higher than global replanning in the majority of cases where global replanning wins.

### Future work.

- A complete (CBS-style) repair search to close the incompleteness gap of prioritized planning.
- Modelling parked/idle robots that remain on the floor and can be re-disrupted.
- Dwell times at pickup/delivery instead of instantaneous service.
- Longer per-robot task lists so that reassignment interacts with queued future work.
- Learned or adaptive negotiation weights instead of the fixed  $w_1$ – $w_4$  used here.
- Validation against real warehouse operation traces rather than a synthetic procedurally generated layout.

## A Repository layout

---

```

backend/app
 domain/ grid.py, robot.py, task.py, disruption.py, metrics.py
 planning/ space_time_astar.py, heuristics.py, reservation_table.py,
 collision_detection.py, prioritized_mapf.py (initial planning only)
 repair/ dispatcher.py, local_repair.py, affected_set.py, conflict_graph.py,
 negotiation.py, task_reassignment.py, repair_validation.py, models.py
 simulation/ engine.py, event_log.py, event_scheduler.py, scenario_loader.py,
 builtin_scenarios.py, scenario_generator.py
 experiments/ configurations.py, batch_runner.py, statistics.py, make_tables.py
 api/ schemas.py, simulation_routes.py, experiment_routes.py,
 websocket.py, state.py
 cli.py, main.py, tests/

frontend (Next.js / TypeScript / Tailwind / Framer Motion / Recharts)
 app/ page.tsx, simulation/page.tsx, experiments/page.tsx
 components/ GridView, RobotSprite, ControlPanel, DisruptionPanel, MetricsPanel,
 Timeline, EventLog, ConflictGraph, RepairInspector, ExperimentCharts
 lib/ api.ts, websocket.ts, types.ts, viz.ts

```

---

The algorithmic engine (domain, planning, repair, simulation, experiments) has no dependency on the web framework or the frontend and is driven identically by the REST/WebSocket API, the CLI, the batch experiment runner, and the pytest suite.

## B Test summary

The backend test suite (python -m pytest) contains 75 tests across `test_milestone1_core.py` through `test_milestone6_8_repair.py`, `test_api.py` and `test_experiments.py`, covering the grid/robot/task domain model, Space-Time A\*, prioritized multi-agent planning, the execution engine and its invariants, cell-blockage handling, and the full repair/negotiation pipeline including fuzz-style stress scenarios.

## C Repair configuration parameters

**Table 7.** RepairConfig default parameters (backend/app/repair/models.py).

| Parameter                              | Default | Role                                                       |
|----------------------------------------|---------|------------------------------------------------------------|
| <i>Local repair strategies</i>         |         |                                                            |
| max_wait                               | 12      | Strategy 1: longest wait inserted at the conflict point    |
| max_shift                              | 12      | Strategy 2: longest whole-suffix delay                     |
| detour_window                          | 14      | Strategy 3: how far ahead a detour may rejoin the old plan |
| <i>Task reassignment, Equation (3)</i> |         |                                                            |
| lambda_load ( $\lambda$ )              | 4.0     | Weight on recipient’s current load                         |
| mu_impact ( $\mu$ )                    | 2.0     | Weight on repair impact on others                          |
| <i>Negotiation bid, Equation (4)</i>   |         |                                                            |
| $w_1$                                  | 1.0     | Weight on completion-time increase $\Delta C$              |
| $w_2$                                  | 0.5     | Weight on extra path length $\Delta L$                     |
| $w_3$                                  | 1.0     | Weight on priority penalty $P$                             |
| $w_4$                                  | 30.0    | Penalty for modifying a previously-untouched robot         |

## D Full result tables

Full per-disruption-type tables (flowtime, modified agents, repair time, messages, affected-set expansions, largest  $|A_d|$ , modified ratio, success rate, and failure reasons, each broken down by all  $5 \times 5$  combinations of agent count and density) are generated by `python -m app.experiments.make_tables` from the raw run data in `backend/experiment_outputs/` and are included as:

- `docs/results/cell_blockage.md`
- `docs/results/robot_breakdown.md`
- `docs/results/emergency_task.md`
- `docs/results/mixed.md`